# KNN

反向链接 · 1　　更新时间 期限

信息

## KNN 算法简介

KNN (K-Nearest Neighbors，K 近邻算法) 是一种**非参数、有监督的学习分类器**，KNN 使用切点来对单个数据点的分组进行分类或预测。虽然也还能算出KNN 可以用于回归或分类问题，但它通常用作分类算法，假设可以在彼此附近找到相似点，对于分类问题，根据多数评分配类别标签，也就是使用在距离最近周围最常表示的标签。

k 近邻算法中距离度量：两个样本的特征距离反映了彼此之间的相似程度，常见的距离度量有欧式距离，马氏距离……老师上课还补充了很多距离度量方法

k-means 算法中 k 值的选择

1. 如果 k 值较小，可通过区域小的近邻来做中决策结果进行预测，因为领域小，代表训练样本与未知样本的特征最为相似，其参数本身对于预测不起作用，所以预测的准确度会高；但缺乏参数的稳定性，若给定的训练样本点发生噪声，难以进行准确预测至少类别，总结：k 值较小，k 近邻模型算法就是复杂，容易导致过拟合。
2. 如果 k 值较大，优点是减少了预测的估计误差，缺点是增大了预测的过拟误差。此时与未知样本较近的训练点也参与预测，有可能产生错误预测，因此 k 值较大会使得简单，将忽略大量的训练样本中的有用信息。

## 数据集介绍

iris sonar 数据集在之前已经介绍过了，不再赘述；Minist、CIFAR 数据集之前也用过，这里也不多说

1. UCM

高分辨率遥感影像数据集。它都广泛用于场景分类的研究。

**内容**：该数据集包含了21个同的土地利用类别，如农业用地、机场、海滩、建筑物等，每个类别有 100 张图片，总共 2100 张图片，每张图片的大小为 256×256 像素，所有图像都经过了人工标注，确保了数据的质量。

2. NWPU

大规模高分辨率遥感图像地景分类数据集

**内容**：这个数据集共有 31 种不同的地物类别，比如教堂、森林、工业区、湖泊等，对于每个类别，数据集中含有 700 张图像，共计 31,500 张图像，这些图像是从 Google Earth 上收集而来的，覆盖了全球多个地区。

**特点**：与 UCM 相比，NWPU-RESISC45 提供了更多的类别和更丰富的样本数量，这有助于训练更加复杂和鲁棒的模型，因此在实际应用中，这个数据集的表现通常会更为出色。

## 算法介绍与设计

KNN 主体算法实现

```
# KNN Algos
# L2 square distance between two vectorized images x and y
def distance2(x,y):
    return np.sum(np.square(x-y))
# L2 distance between two vectorized images x and y
def distance2(x,y):
    return np.sqrt(np.sum(np.square(x-y)))
def distance3(x,y):
    return np.linalg.norm(x-y)

def kNN(x, k, data, label):
    data = np.asarray(data)
    label = np.asarray(label)
    x = np.asarray(x)
    distances =[distance3(x,data[i]) for i in range(len(data))]
    idx = np.argsort(distances,k).astype(np.int64)
    clas, freq = np.unique(label[idx[:k]], return_counts=True)
    return clas[np.argmax(freq)]
```

- 定义距离函数，但随机试图离写 L2 距离
- kNN 算法中计算 k 到训练数据的最近的样本，找到前k个最小距离的索引，统计前k个最近邻的标签及其出现的次数。
- 最后返回出现次数最多的标签作为预测结果

## 系统设计

因为老师给的网站不是很稳定，对于数据集我统一使用 huggingface 提供的 datasets，其还提供了切分方法以及打乱、train_test_split 以及 shuffle

```
from datasets import load_dataset

ds = load_dataset("pickit-ieee/iris", split="train")

ds = ds.train_test_split(test_size=0.2, shuffle=True)

ds
```

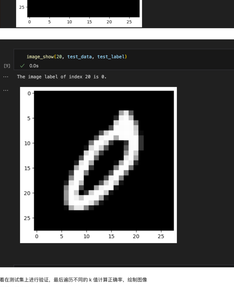
[17] ✓ 2.8s 模型 Data Wrangler 中打开"ds"　　Python

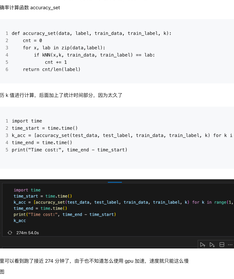
```
DatasetDict({
    train: Dataset({
        features: ['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm',
        num_rows: 120
    })
    test: Dataset({
        features: ['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm',
        num_rows: 30
    })
})
```

然后简单定义了一个 image_show 函数来查看数据集中内容

```
def image_show(i, data, label):
    x = data[i] # get the vectorized image
    x = np.asarray(x)
    x = x.reshape((28,28)) # reshape it into 28x28 format
    print("The image label of index %d is %d." %(i, label[i]))
    plt.imshow(x, cmap='gray') # show the image
```

```
image_show(100, train_data, train_label)
```



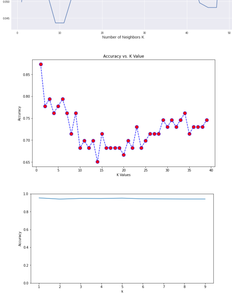

接着在原试集上进行验证，最后画历不同的 k 值计算正确率，绘制图像

正确率计算函数 accuracy_set

```
def accuracy_set(data, label, train_data, train_label, k):
    cnt = 0
for x, lab in zip(data,label):
    if kNN(x,k, train_data, train_label) == lab:
        cnt += 1
return cnt/len(label)
```

遍历 k 值进行计算，后面加上了统计时间部分，因为太久了

```
import time
time_start = time.time()
k_acc = [accuracy_set(test_data, test_label, train_data, train_label, k) for k i
time_end = time.time()
print("Time cost:", time_end - time_start)
```




这里可以看到跑了接近 274 分钟了，当个也不知道怎么使用 gpu 加速，速度就只能这么慢

绘图

```
X = [k for k in range(1,10)]
plt.figure(figsize = (10,5))
plt.xlabel("k")
plt.ylabel("Accuracy")
plt.xticks(X)
plt.plot(X,k_acc)
plt.savefig("k_acc.png")
```

## 错误率分析

前两个数据集还好一点，不需要多久，本来打算全都做的，但最后只做了前三个数据集

## 参考链接：

https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

https://scikit-learn.org/stable/modules/clustering.html#k-means

https://huggingface.co/docs/datasets/tutorial

## 问题汇总与收获

- 学习了 KNN 算法，并且了解了 K-Means 聚类方法（因为最开始写错了，白忙活了）
- 通过了 huggingface datasets 库的使用，了解加载数据集的优缺点

https://github.com/little13/pattern_recognition/tree/main/KNN

目录架构：

1. comp_dim.ipynb：根据论文和不同特征维度训练并绘图
2. minist.py sonar.py iris.py：主入口
3. test.ipynb：单元测试文件（写的很乱）

启动代码示例

```
python dis_install.py

python minist.py
```

注：使用 huggingface_hub 下载的 datasets 在~/.cache/huggingface/datasets 下，实验完记得清理

Bi-Directional Links　　Show